Checklist de Entrega — Obra Fácil

> **Versão:** 27 de abril de 2026
> **Baseado em:** `docs/auditoria_requisitos_professor.md`
> **Legenda:** ✅ Atendido | ⚠️ Parcial | ❌ Ausente | 🔧 Ação pendente

Discovery

Definição do problema

- [x] `docs/01-produto/definicao_problema.md` existe
- [x] Problema está claro e conciso (seção 1 do arquivo)
- [x] Persona está definida (Carlos Alberto, 45 anos, proprietário de imóvel)
- [x] Objetivo do produto está definido (seção 3 do arquivo)
- [x] Contexto do problema está descrito
- [x] Solução atual do usuário está descrita (indicações e sites genéricos)
- [x] Impacto da dor está documentado (tempo, insegurança, custo)
- [ ] 🔧 Evidência de deepresearch/validação externa do problema

Artefatos adicionais de Discovery (bônus)

- [x] `docs/01-produto/persona.md` — detalhamento rico da persona
- [x] `docs/01-produto/jornada_do_usuario.md` — jornada mapeada
- [x] `docs/01-produto/lean_canvas.md` — modelo de negócio
- [x] `docs/01-produto/user_stories.md` — histórias de usuário por perfil

Refinamento

PRD (`docs/01-produto/prd.md`)

- [x] Arquivo existe
- [x] Descrição do produto (problema, solução, público-alvo)

- [x] Diferenciais do produto (3 diferenciais documentados)
- [x] Perfis de usuário (Cliente, Profissional, Lojista)
- [x] Principais funcionalidades com critérios de aceite (RFN-01 a RFN-04)
- [x] Requisitos não funcionais (RNF-01 a RNF-03)
- [ ] ⚠️ RNFs desatualizados — apenas 3; implementação tem ~8 RNFs reais
- [x] Métricas de sucesso
- [x] Premissas e restrições
- [x] Escopo por versão (v1 MVP, v2, v3)

Especificação de Requisitos (`docs/spec_req.md`)

- [ ] 🔧 Arquivo `docs/spec_req.md` standalone não existe
- [ ] 🔧 Requisitos funcionais com IDs (RF-01 a RF-NN) não documentados formalmente
- [ ] 🔧 Requisitos não funcionais com IDs (RNF-01 a RNF-NN) não expandidos
- [ ] ⚠️ *Conteúdo parcialmente disponível no PRD como RFN/RNF*

Especificação Técnica (`docs/02-arquitetura/spec_tech.md`)

- [x] Arquivo existe
- [x] Visão geral técnica e público-alvo do documento
- [x] Arquitetura de referência (estilo, componentes, protocolos)
- [x] Stack Frontend (Next.js, TypeScript, Tailwind)
- [x] Stack Backend (NestJS, Node.js, TypeScript)
- [x] Stack de desenvolvimento (IDE, pacotes, Docker, CI/CD)
- [x] Segurança (autenticação, RBAC, criptografia, DevSecOps)
- [x] APIs (endpoint, versionamento, autenticação)
- [x] Tenancy (isolamento por usuário, RLS)
- [x] Diretrizes para IA
- [ ] ⚠️ Stack cita Prisma como ORM (projeto usa `pg` direto) — ver P1-05
- [ ] ⚠️ Stack cita Firebase/Supabase Auth (projeto usa Clerk) — ver P1-05

Especificação de UI (`docs/03-design-ux/spec_ui.md`)

- [x] Arquivo existe
- [x] Interfaces gráficas INT-01 a INT-04 com campos, botões, links
- [x] Considerações de UX em cada interface
- [x] Fluxo de navegação (4 etapas do cliente)
- [x] Diretrizes para IA
- [ ] ⚠️ Spec cobre apenas 4 telas do MVP; ~15 telas implementadas não documentadas

Design

- [x] `docs/03-design-ux/design_system.md` existe com tipografia, paleta e componentes
- [ ] ⚠️ Protótipos/mockups sem imagens versionadas no repositório
- [ ] 🔧 `docs/modelo_dados.md` não existe como documento dedicado
- [x] DDL completo disponível em `docker/01-schema.sql` (15+ tabelas)
- [x] Diagrama de arquitetura em `docs/02-arquitetura/arquitetura.md`
- [x] Diagramas C4 em `docs/diagramas/` (context, container, component, deploy)

Delivery

Preparação do ambiente

- [x] `README.md` existe com instruções de setup
- [x] `docs/04-ambiente-e-processos/setup-local.md` com passos detalhados
- [x] `docs/04-ambiente-e-processos/GUIA_DESENVOLVIMENTO_LOCAL.md`
- [x] `apps/backend/.env.example` existe
- [x] `apps/frontend/.env.local.example` existe
- [ ] ⚠️ `.env.example` único na raiz do monorepo não existe (roteiro orienta um único arquivo)
- [x] `.gitignore` protege `.env` e `.env.local`

Estrutura do projeto

- [x] Monorepo npm workspaces configurado
- [x] `apps/frontend/` — Next.js 15 com App Router
- [x] `apps/backend/` — NestJS 11 com 13 módulos
- [x] `packages/shared/` — tipos e schemas Zod compartilhados
- [x] Frontend e backend completamente separados
- [x] Comunicação exclusiva via API REST (`lib/api/client.ts`)
- [x] Frontend sem `@supabase/supabase-js`

Docker

- [x] `docker-compose.yml` existe com 3 serviços (db, backend, frontend)
- [x] `apps/backend/Dockerfile` — multi-stage build

- [x] `apps/frontend/Dockerfile` — multi-stage build
- [x] Serviço `db` usa PostgreSQL 17
- [x] Ambiente local sobe completamente com `docker compose up`

Autenticação (Clerk)

- [x] `ClerkAuthGuard` no backend valida JWT Clerk
- [x] Bypass seguro para desenvolvimento (`DISABLE_CLERK_AUTH=true` + `X-Dev-User-Id`)
- [x] Bypass bloqueado em produção (`process.exit(1)` se NODE_ENV=production)
- [x] Sincronização de perfil via JIT provisioning no guard
- [x] Frontend usa `@clerk/nextjs` (componentes oficiais de login)
- [ ] ⚠️ Roteiro 2.3.3 pede frontend sem dependência Clerk — divergência arquitetural documentada (ver auditoria seção 16)

Supabase

- [x] Frontend não acessa Supabase diretamente
- [x] Banco PostgreSQL (Supabase em produção) acessado apenas pelo backend
- [x] Driver `pg` (node-postgres) via `DatabaseService`
- [x] RLS habilitado no banco (`docker/04-rls.sql`)

Testes

Execução de testes

- [x] `npm run lint --workspace=backend` — executa sem erros
- [x] `npm run lint --workspace=frontend` — executa sem erros
- [x] `npm run build --workspace=@obrafacil/shared` — compila
- [x] `npm run build --workspace=backend` — compila
- [x] `npm run build --workspace=frontend` — compila (Next.js)
- [x] `npm test --workspace=backend` — ~55 testes passando (Jest)
- [x] `npm run test --workspace=frontend` — ~9 testes passando (Vitest)
- [x] Playwright E2E — fluxos cliente e profissional

Cobertura de testes

- [x] Unitários backend: ClerkAuthGuard, professionals, visits, works, reviews (~49 testes)

- [x] E2E backend: isolamento de usuário, pedidos (~6 testes)
- [x] Unitários frontend: componentes de UI (~9 testes)
- [x] E2E Playwright: busca pública, agendamento, fluxo profissional (~4+ cenários)
- [ ] 🔧 `docs/testes.md` não existe — plano e evidências não documentados formalmente

Liberação

Pipeline CI/CD

- [x] `.github/workflows/ci.yml` existe e é funcional
- [x] CI roda em `push` e `pull_request` para `main` e `develop`
- [x] Jobs paralelos: quality (lint + typecheck) e unit-tests
- [x] Job de build E2E após quality + unit-tests
- [x] Badge de CI no `README.md`
- [x] `.github/workflows/deploy.yml` existe
- [x] Deploy com `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`
- [ ] ⚠️ Trigger `push` no `deploy.yml` comentado — deploy apenas manual

Vercel

- [x] Projeto frontend configurado no Vercel
- [x] Projeto backend configurado no Vercel (via `api/index.ts`)
- [x] `apps/frontend/vercel.json` existe
- [x] `apps/backend/vercel.json` existe
- [x] Deploy documentado em `docs/diagramas/deploy.md`

Documentação de entrega

- [x] README com badges de status, stack e instruções
- [x] `docs/04-ambiente-e-processos/variaveis-ambiente.md` com todas as variáveis
- [x] `docs/04-ambiente-e-processos/setup-local.md` com setup local
- [x] `.fluxo/` com roteiros do professor — **criado nesta auditoria**
- [x] `docs/auditoria_requisitos_professor.md` — **criado nesta auditoria**
- [x] `docs/plano_correcao_requisitos_professor.md` — **criado nesta auditoria**
- [x] `docs/checklist_entrega_professor.md` — **este arquivo**

Evidências

- [x] Repositório público no GitHub: `https://github.com/lexcesar/obra-facil`
- [x] CI badge no README referenciando `ci.yml`
- [ ] 🔧 URL de produção frontend não documentada no README
- [ ] 🔧 URL de produção backend (Swagger) não documentada no README
- [ ] 🔧 Credenciais de teste (usuario/senha) não documentadas no README
- [ ] 🔧 Screenshots das telas principais ausentes

Resumo de ações pendentes

Prioridade	Item	Estimativa
P1	Criar `docs/spec_req.md`	1h
P1	Criar `docs/testes.md`	30min
P1	Criar `docs/modelo_dados.md`	45min
P1	Habilitar/documentar deploy automático	5min
P1	Corrigir stack em `spec_tech.md` (ORM/Auth)	15min
P2	Screenshots de telas no `docs/03-design-ux/screenshots/`	30min
P2	`.env.example` único na raiz	10min
P2	Criar `docs/api.md`	45min
P2	Documentar URLs de produção e credenciais de teste no README	15min
P2	Expandir RNFs no PRD	20min